

计算概论A（实验班）

函数式程序设计

胡振江，张伟

计算机学院

2026年9月9日

授课教师



胡振江

- 1988: 上海交通大学 计算机系 本科毕业
- 1996: 日本东京大学 信息工学 博士学位
- 1997: 日本东京大学 工学部 讲师
- 2000: 日本东京大学 工学部 副教授
- 2008: 日本国立信息学研究所 教授
- 2018: 日本东京大学 信息科学技术学院 教授
- 2019: 北京大学 计算机学院 讲席教授

日本工学会会士、IEEE Fellow、ACM杰出科学
日本工程院院士、欧洲科学院院士



研究简介

- Functional Programming (1985-now)
 - Calculating Efficient Functional Programs
- Programming Languages (1992-now)
 - Parallel programming languages
 - Domain Specific Languages/Language engineering
 - C*: A System PL Integrating Coding and Proving
- Bidirectional Languages (2003-now)
 - Bidirectional languages for system/data interoperability



胡振江

职称：教授

研究所：软件研究所

研究领域：程序设计语言，函数式语言，软件工程，程序演算

燕园校区：

理科1号楼1247室

昌平新校区：

计算机大楼449室

办公电话：86-10-62757974

电子邮件：huzj@pku.edu.cn

个人主页：<http://sei.pku.edu.cn/~hu>



张伟

职称：副教授

研究所：软件研究所

研究领域：群体软件开发，群体智能系统设计

电子邮件：zhangw.sei@pku.edu.cn

燕园校区：

理科1号楼 1803室

昌平新校区：

计算机大楼 437室

- 助教:

- 余泰来 yutailai@stu.pku.edu.cn

- 庄嘉毅 wybxc@stu.pku.edu.cn



课程微信群

群聊: 计算概论实验班 2026
(staff)



该二维码7天内(9月13日前)有效, 重新进入将更新

课程网页

<https://zhenjiang888.github.io/FP/2026/>



函数式程序设计 / 2022秋

新通知

- 2022/11/4 讲义: 计算机的基本原理

课程简介

函数式程序设计是一种具有悠久历史的程序设计方法。在几十年以来的大多数时间中，函数式程序设计栖身于象牙塔内，慢慢成长，积蓄力量，并没有在软件开发实践中得到广泛使用。世异时移，这种情况正在发生变化。随着软件系统复杂性的不断提高以及软件与现实世界关系的日益紧密，如何严格确保软件系统的正确性和可靠性，逐渐成为一个重要的现实问题。函数式程序设计建立在严格的数学概念的基础上，具有良好的数学性质，为上述问题提供了一种现实可行的解决方案。目前，大多数的主流程序设计语言都在逐渐引入函数式程序设计的各种成分。本课程将系统介绍函数式程序设计的基本思想、核心概念、以及相关的程序设计方法和程序推理方法，为学生从事算法设计、程序语言设计、软件开发等领域的研究和实践工作建立坚实的理论基础。

课程信息

理论课程：第二教学楼 202；每周周三5~6节、每周周五3~4节
上机实习：理科一号楼 1235；每周周五5~6节

课程大纲

课程基本目的

培养学生使用函数式思维进行程序设计和推理的基本能力，了解函数式程序设计语言的基本运行原理，熟练使用Haskell语言及相关工具进行程序设计与调试，熟练使用形式化方法对Haskell程序进行推理和变换。



课程讲义

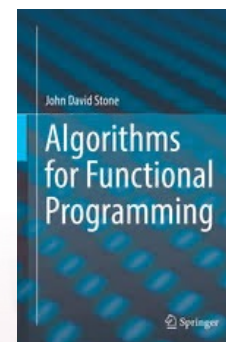
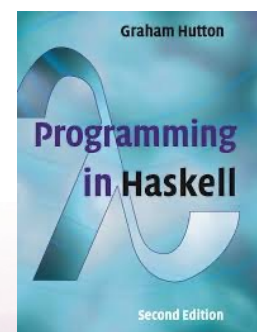
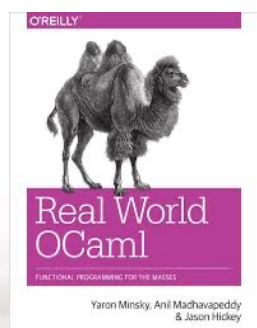
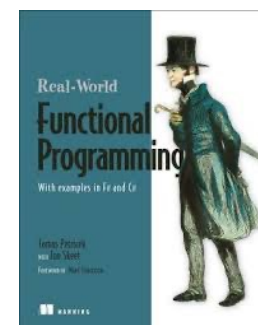
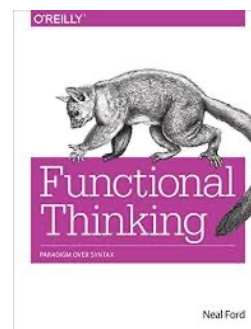
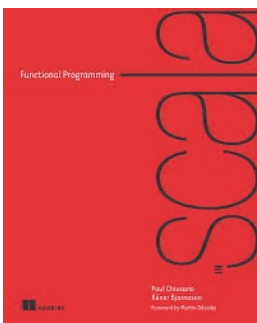
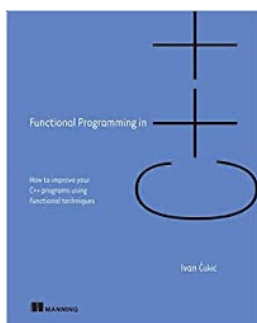
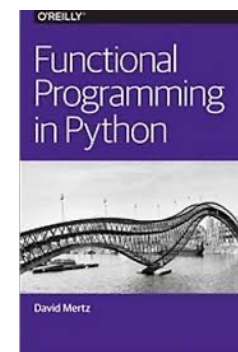
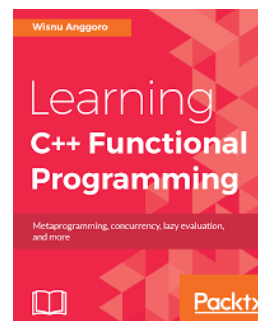
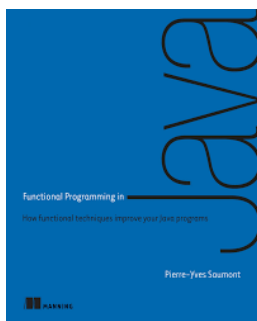
在这里可以下载课程讲义。

2022/09/07	胡振江： 导言：函数式程序设计 [幻灯片] [作业] 第一章：概论 [幻灯片]
2022/09/09	张伟： 第2.1章：初见函数式思维 [幻灯片]
2022/09/14	张伟： 第2.1章：初见函数式思维——若干补充说明 [幻灯片] [作业] 第2.2章：初见Haskell [幻灯片]
2022/09/16	张伟： 第2.2章：初见Haskell [幻灯片] [作业]
2022/09/21	张伟： 第3章：类型与类簇 [幻灯片] [作业] 第4章：函数的定义 [幻灯片]
2022/09/23	张伟： 第5章：List Comprehension [幻灯片] [作业] 第6章：递归函数 [幻灯片]
2022/09/28	张伟： 第7章：高阶函数 [幻灯片] [作业] 第8章：类型和类簇的声明/定义 [幻灯片]
2022/09/30	张伟： 第8章：类型和类簇的声明/定义 [幻灯片]
2022/10/05	张伟： 第9章：An Example: The Countdown Problem [幻灯片] [作业] 第10章：交互式程序设计 [幻灯片]



学习函数式程序设计的N个理由

函数式程序设计… 火了 (2020) ?



AI时代还需要学习函数式编程?

- 计算思维

- 什么是计算：算数？状态转换？推理？物理过程？
- 描述计算，构造计算，分析计算

- 函数式编程不仅仅是一种“编码风格”，而且是一种“推理框架”

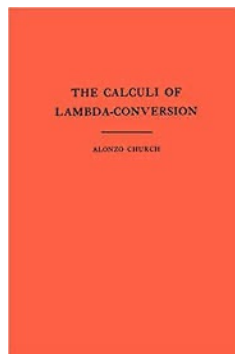
- 清晰的问题描述能力
- 强推理能力
 - 等式推理
 - 局部推理
 - 编译即证明

函数式语言: 讨论“计算”的鼻祖语言



Alonzo Church (1903-1995)

Lambda Calculus
λ演算



Church-Turing Thesis
邱奇-图灵论题

*If an algorithm exists,
then there is an equivalent
Turing Machine or
applicable Lambda-function
for that algorithm.*

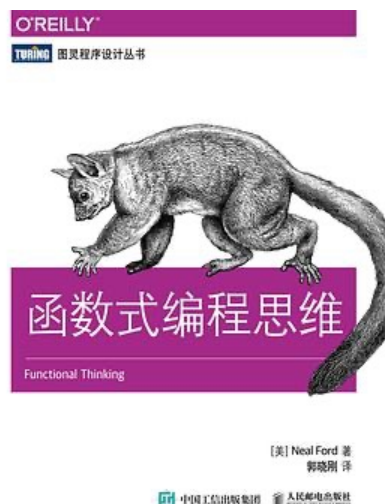


John McCarthy (1927-2011)

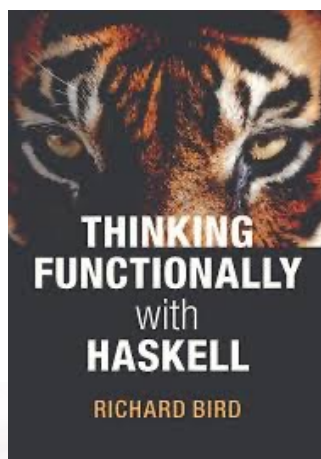
Father of AI and Lisp

Operators + Notions for Functions
=
Whole Programming Languages

适合现代软件开发的函数式编程思维



复合形: 从简单到复杂, 模块化
抽象性: 将琐碎的细节交给运行时
简洁透明性: 不是封装不确定因素,
而是减少不确定因素



程序设计
=
正确而简明的实现
+
基于程序推理的优化

很多顶级大学采用的第一门程序设计课

We aim to teach the core principles so that students can quickly grasp any new language that comes along.

- 剑桥大学
- 牛津大学
- 东京大学
- 爱丁堡大学
- Chalmers University of Technology
- University of New South Wales
- Australian National University

Dijkstra呼吁在Austin大学继续教授FP

To the members of the Budget Council

I write to you because of a rumour of efforts to replace in the introductory programming course of our undergraduate curriculum the functional programming language Haskell by the imperative language Java, and because I think that in this case the Budget Council has to take responsibility lest the decision be taken at the wrong level.

You see, it is no minor matter. Colleagues from outside the state (still!) often wonder how I can survive in a place like Austin, Texas, automatically assuming that Texas's solid conservatism guarantees equally solid mediocrity. My usual answer is something like "Don't worry. The CS Department is quite an enlightened place, for instance for introductory programming we introduce our freshmen to Haskell"; they react first turns out that their und from Pascal to something

A very practical r course is that most stud programming. Facing the drives home the message quickly they will observ that are very hard (or i their high school days.

A fundamental reason for the preference is that functional programs are much more readily appreciated as mathematical objects than imperative ones, so that you can teach what rigorous reasoning about programs amounts to. The additional advantage of functional programming with "lazy evaluation" is that it provides an environment that discourages operational reasoning.

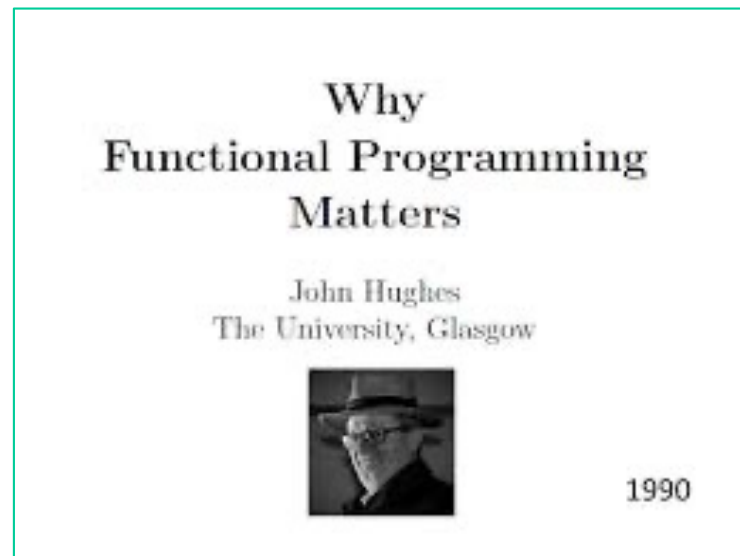
Finally, in the specific comparison of Haskell versus Java, Haskell, though not perfect, is of a quality that is several orders of magnitude higher than Java, which is a mess (and needed an extensive advertizing campaign and aggressive salesmanship for its commercial acceptance). It is bad enough that, on the whole, in-

dustry accepts designs of well-identified lousiness as "de facto" standards. Personally I think that the University should keep the healthier alternatives alive.

It is not only the violin that shapes the violinist, we are all shaped by the the tools we train ourselves to use, and in this respect programming languages have a devious influence: they shape our thinking habits. This circumstance makes the choice of first programming language so important. One would like to use the introductory programming course as a means of creating a culture that can serve as a basis for a computing science curriculum, rather than be forced to start that with a lot of unlearning (if that is possible at all: what has become our past, forever remains so). The choice implies a grave responsibility towards our undergraduate students, and that is why it can not be left to a random chairman of something but

It is not only the violin that shapes the violinist, we are all shaped by the tools we trans ourselves to use, and in the respect programming languages have a devious influence: they shape our thinking habits.

FP专家说的理由



Volume 2, Issue 3
September 2015

Article Contents

Abstract

INTRODUCTION

CORRECTNESS OF PROGRAM
CONSTRUCTION

STRUCTURING COMPUTATION

PARALLEL AND DISTRIBUTED
COMPUTATION

FUNCTIONAL THINKING IN
PRACTICE

How functional programming mattered

Zhenjiang Hu , John Hughes, Meng Wang

National Science Review, Volume 2, Issue 3, September 2015, Pages 349–370,
<https://doi.org/10.1093/nsr/nwv042>

Published: 13 July 2015 Article history ▼

 PDF  Split View  Cite  Permissions  Share ▼

Abstract

In 1989 when functional programming was still considered a niche topic, Hughes wrote a visionary paper arguing convincingly ‘why functional programming matters’. More than two decades have passed. Has functional programming really mattered? Our answer is a resounding ‘Yes!’. Functional programming is now at the forefront of a new generation of programming technologies, and enjoying increasing popularity and influence. In this paper, we review the impact of functional programming, focusing on how it has changed the way we may construct programs, the way we may verify programs, and fundamentally the way we may think about programs.

Keywords: [functional programming](#), [functional languages](#), [equational reasoning](#), [monad](#), [high order function](#)

John Hughes: **Why Functional Programming Matters**. Comput. J. 32(2): 98-107 (1989)

Zhenjiang Hu, John Hughes, Meng Wang: **How functional programming mattered**. *National Science Review*, Volume 2, Issue 3, September 2015, Pages 349–370



目次

立体をどうとらえるか

線形計画法

生命科学と数理工学

計算機プログラムの数理

— 列の上のアルゴリズム

プログラム演算の数理

— 数式演算による並列プログラミング

カオスとコイン投げ

携帯電話はどうしてつながるのか—携帯電話ネットワークの頂点

彩色問題

携帯電話に日本語を入力するには—自然言語の数理

統計モデルの数理

数値計算とコンピューター—中間値の定理の誤差解析への応用

情報はどうやって守るか—情報セキュリティと秘密分散法

時系列解析と揺動散逸原理

混合行列の話

相手を思い通りに動かす技術

数理を社会に適合させるには? —統計学を例に

日本評論社 (2002/09 発売)

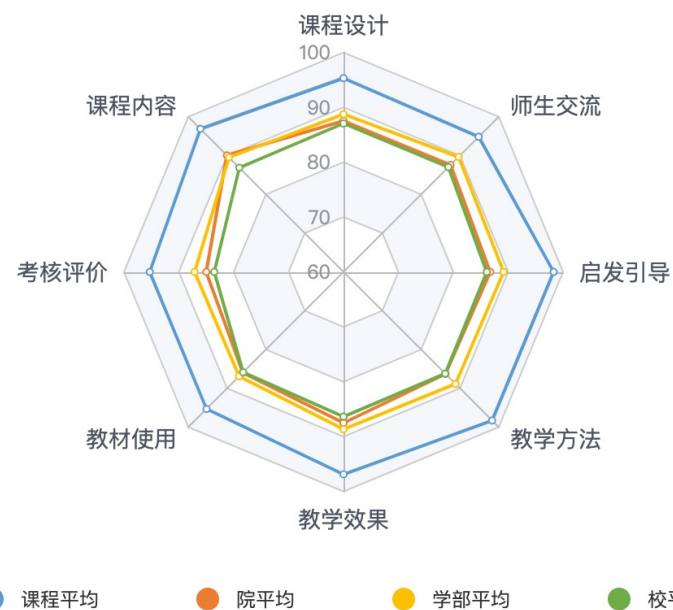


本课程授课状况

2020年：20人
2021年：45人
2022年：46人
2023年：64人
2024年：80人
2025年：70人

2021-2022-1 学期期末学生课程评估结果雷达图

课程名称：计算概论A（实验班） 教学班号：1 课程号：04830530 开课院系：信息科学技术学院





拔尖班计算概论教学的一点尝试

胡振江 张伟 《计算机教育》

2025年第1期9-10

课程目的

课程目的

- 通过教授函数式程序设计的基本概念（例如递归，抽象，高阶函数和数据类型，程序推理）并说明其实际使用，使学生能够
 - 熟悉程序设计的基本技术和函数式思维方式
 - 理解“计算”的基本概念
 - 培养学生对程序设计问题的分析和解决能力



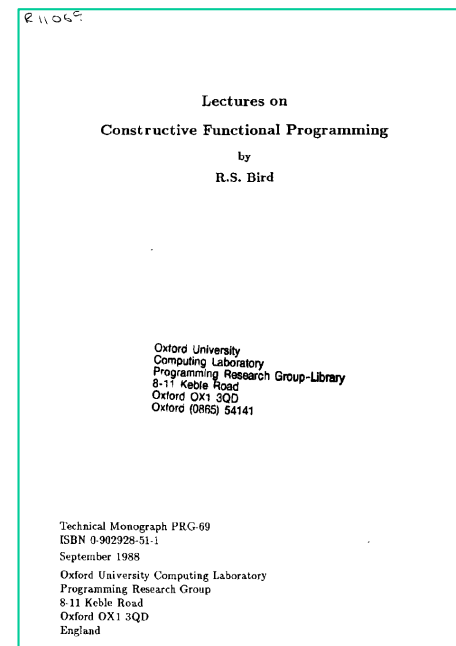
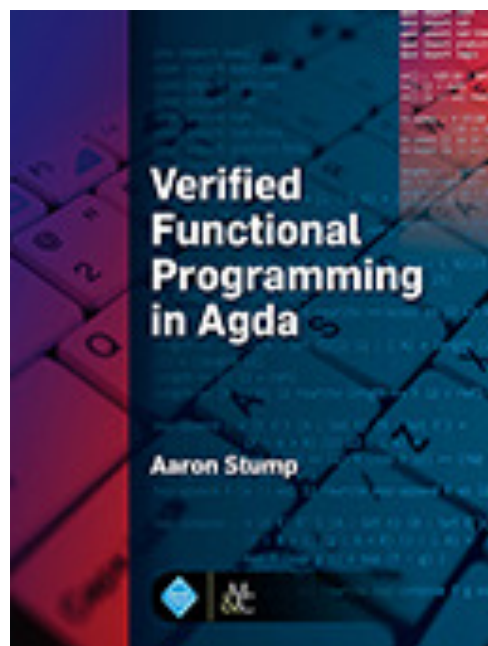
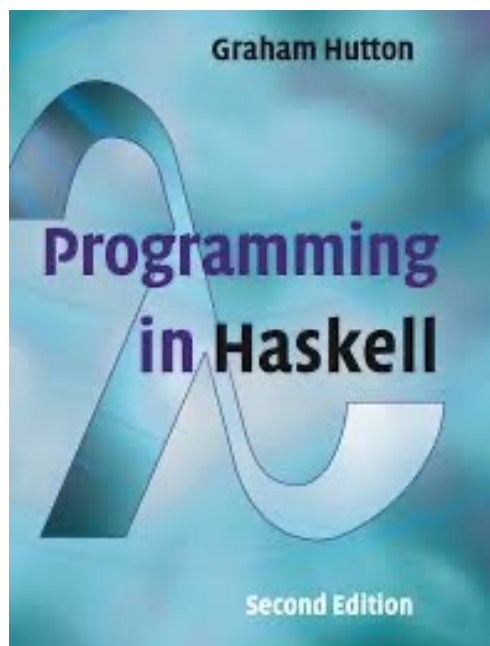
选课的同学要求具备良好的数学基础
有一定的编程经验

教学内容

主要包括三部分

- Haskell 函数式程序设计: 约11次课
 - 类型, 函数定义, 递归定义, 高级函数, 类型定义, 描述副作用的函数, 惰性计算, 各种应用
- (基于Agda的) 程序推理与演算: 约10次课
 - 程序的结构化
 - 程序推理与演算理论
 - 程序推导与程序综合
- 计算概论大课: 6次课

主要教材



Graham Hutton, **Programming in Haskell**, Cambridge University Press, 2016.

Aaron Stump, **Verified Functional Programming in Agda**, ACM and Morgan & Claypool, 2016

Richard Bird, **Lecture Notes on Constructive Functional Programming**, Technical Monograph PRG-69, Oxford University, 1988.

评分标准

评分标准

- 平时：30分（每周三中午前交一次作业）
 - 期中：30分
 - 期末：40分
-
- 期中考试：2026年11月4日 5-6节
 - 期末考试：2026年12月30日下午